

Java – Beautifully Broken

Java puts its threads on a leash, and Spring finally retires RestTemplate

Issue #5

Fri, 09 Oct 2026

~38 min read

Level 3/5

Today's two stories

1. **Structured concurrency is about to become a permanent part of Java.** JEP 543 proposes finalising `StructuredTaskScope` in JDK 28 with no changes from JDK 27, after two incubators and seven previews. The code that removes the preview flag is already under review. It gives you a way to run several tasks in parallel that can't leak threads or silently keep running after a failure.
2. **RestTemplate is now officially deprecated for removal.** Spring Framework 7.1 marks the HTTP client that most Spring codebases have used for 15 years with `@Deprecated(forRemoval = true)`, and Spring Boot 4.2 deprecates `RestTemplateBuilder` and `TestRestTemplate` with it. Both go GA in November. Here's what that means for your build, your tests and your migration plan.
3. **On the radar:** 6 more things that happened, listed at the end.

STORY 1 Structured concurrency: Java's parallel tasks get a parent

JDK 28 · OPENJDK · PROJECT LOOM · JEP 543 CANDIDATE, IMPLEMENTATION UNDER REVIEW (OCT 2026) · ~16 MIN READ

Here's a bug that has probably shipped in your codebase. A request handler fires off two calls in parallel: one for the user, one for their orders. The user call fails after 200 milliseconds. Your handler returns an error straight away, right? It doesn't. It waits, quietly, for the orders call to finish a full second later, and then throws away the result. Nobody wrote that behaviour on purpose. It's simply what Java's concurrency tools do when you don't tell them otherwise. After four years of experiments, Java is about to ship a standard fix.

WHAT HAPPENED

In September, **JEP 543, "Structured Concurrency"**, was promoted from a draft to a **Candidate** *≈ a JEP that the OpenJDK community has accepted as worth doing, but that hasn't yet been assigned to a specific release*. InfoQ's roundup described its aim as letting Java "treat groups of related tasks running in different threads as a single unit of work", and reported that it would finalise the feature with no changes.¹ An open-source project tracking the JEP noted it "was a Candidate as of 2026-09-17" and summarised the proposal as finalising structured concurrency "in JDK 28 without change from JDK 27".²

The implementation is already in review. Pull request #32602 in the OpenJDK repository, titled "8391460: Implement JEP 543: Structured Concurrency" and opened by Alan Bateman, makes the `StructuredTaskScope`

API permanent: it removes the preview annotation, sets the `@since` tags to 28, and stops the tests from enabling preview features. Its other changes are documentation polish. As of this week the PR is still open, with a **CSR** *≈ Compatibility and Specification Review: OpenJDK's formal sign-off for any change to a public Java API* (JDK-8391461) not yet approved.³

One precise point on status: JEP 543 isn't yet *targeted* to JDK 28. It wasn't in InfoWorld's 24 September list of JDK 28 features, and Inside.java's "targeted to JDK 28" announcements up to 6 October don't include it.^{4,5} What has happened is that the proposal, the plan and the code all point at JDK 28, the non-LTS release due in March 2027.⁶

Why is a status change on a library API a big deal? Because it ends one of the longest-running previews in Java's history. Once it's final, you can use it without the `--enable-preview` flag, which means libraries and frameworks can finally depend on it.

THE BACKGROUND YOU NEED

Threads, futures and the problem with "fire and forget"

A **thread** *≈ an independent path of execution inside your program; several threads can run at the same time* is how Java does several things at once. For years the usual way to run work on other threads has been an **ExecutorService** *≈ a JDK object that owns a pool of threads and runs the tasks you submit to it*. You `submit()` a task and get back a **Future** *≈ a handle to a result that will be available later; `get()` blocks until it's ready*.

That model has a hidden flaw: the executor has no idea that your two tasks belong together. Look at this code. It's the natural way to write "fetch two things in parallel" in Java 21:

```
// Works on JDK 21+. Both tasks run on virtual threads.
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    Future<String> user    = executor.submit(() -> findUser());
    Future<String> orders = executor.submit(() -> fetchOrders());
    return new Dashboard(user.get(), orders.get());
}
```

Now suppose `findUser()` throws after 200 ms and `fetchOrders()` takes a second. `user.get()` throws, so we leave the `try` block. But closing the executor waits for every task it started, so we're stuck there until `fetchOrders()` finishes, even though its result is now useless. We compiled and ran exactly this scenario on JDK 21: the method failed after **1,027 ms**, and the orders call ran to completion. If you swap the order of the two `get()` calls, the behaviour changes again, because you only notice a failure when you happen to ask for that particular result.

It gets worse without try-with-resources. If you use a long-lived shared executor (the Java 8 habit), nothing waits at all: the orphaned task keeps running in the background after your method has returned. That's a **thread leak** *≈ a thread that keeps running or waiting after the code that started it has finished and forgotten about it*. Multiply it by thousands of requests and you have wasted connections, wasted CPU and very confusing logs.

The HappyCoders guide to the feature lists the full set of problems: a failure in one subtask doesn't stop the others; cancelling the parent doesn't cancel the children; "first result wins" needs complicated code; and in a **thread dump** *≈ a snapshot of what every thread in a running JVM is doing, used for debugging* you just see generic `pool-X-thread-Y` names, with nothing linking a worker thread to the request that started it. The author calls this "unstructured concurrency".⁷

The idea: concurrency shaped like your code

Structured concurrency *≈ a style of concurrent programming in which tasks started inside a block of code must finish before that block ends, just as a method call finishes before the caller continues* borrows a principle you already rely on every day. When method A calls method B, B finishes before A continues. Errors from B flow back to A. You can read the code top to bottom and know what's running.

Structured concurrency applies the same rule to threads. You open a **scope** *≈ a block of code that owns a group of concurrent tasks and doesn't end until all of them have ended*, start some **subtasks** *≈ the pieces of work forked inside a scope, each running on its own thread* inside it, and wait for them at a single point. When the block ends, all of its threads have ended too. If one subtask fails, the scope can cancel the rest. The relationship between parent and children is visible in the code, and, as we'll see, in the thread dump as well.

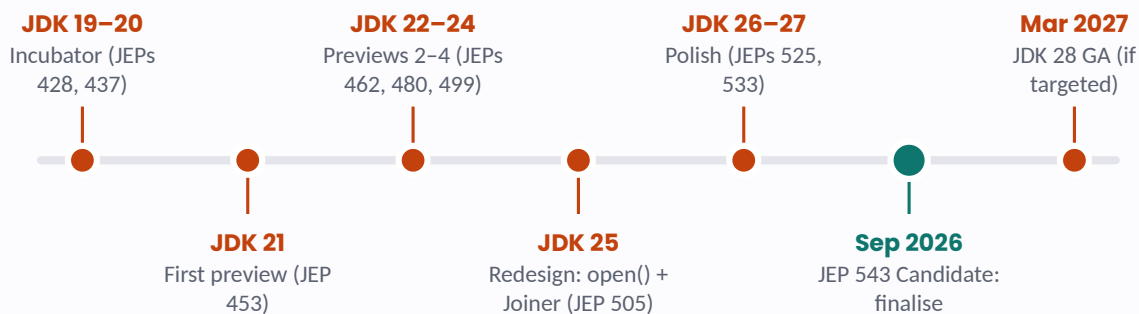
Why it waited for virtual threads

This only became practical once threads got cheap. A classic Java thread, now called a **platform thread** *≈ a Java thread backed one-to-one by an operating-system thread; expensive, so you typically pool a few hundred*, is too expensive to create per subtask. **Virtual threads** *≈ lightweight threads managed by the JVM rather than the OS; you can create millions, so you no longer need to pool them* changed that. They became final in JDK 21 through JEP 444, as part of **Project Loom** *≈ the OpenJDK project that delivered virtual threads, scoped values and structured concurrency*.⁸ With a thread per subtask costing almost nothing, "start a thread for each piece of work and throw it away afterwards" becomes the obvious design. Structured concurrency gives that design rules.

Four years and nine rounds

Structured concurrency has been in the JDK for a long time, just never as a finished API. New Java features usually arrive first as an **incubator** *≈ an experimental module, outside java.base, used to gather early feedback on an API* and then as a **preview API** *≈ a complete but not yet permanent API that you can only use with the `--enable-preview` flag, so it can still change*. This one went through two incubator rounds and seven previews, and the design changed a lot along the way.^{7,1}

Structured concurrency's road to a final API



Sources: HappyCoders version table; InfoQ roundups; Inside.java. JEP 543 is a Candidate, not yet targeted.

In the JDK 21 preview you created a scope with a constructor and chose its policy by picking a subclass. Oracle's JDK 21 guide shows `new StructuredTaskScope.ShutdownOnFailure()`, which "captures the first exception thrown by one of its subtasks, then invokes the shutdown method", and its sibling `ShutdownOnSuccess`, which stops everything once one subtask succeeds.⁹

The fifth preview in JDK 25 (JEP 505) was the big redesign. Constructors were replaced by static factory methods: `new StructuredTaskScope<>()` became `StructuredTaskScope.open()`, and a custom completion policy is now passed in as a separate object, a `Joiner`.¹⁰ Then came two rounds of refinement. JDK 26 (JEP 525) renamed `anySuccessfulResultOrThrow()` to `anySuccessfulOrThrow()`, made `allSuccessfulOrThrow()` return a plain `List` and added a timeout callback.⁷ JDK 27 (JEP 533), targeted in May, cleaned up exceptions and types, which we'll get to in a moment.^{11,12}

Not everyone loved every round. In September 2025, Adam Warski of SoftwareMill published a critique of the fifth preview. He praised its guarantee that scopes don't finish until their forks do, but argued that the scope's main body isn't interrupted when a subtask fails, that ending a scope early needs "tricks", and that `Subtask.get()` is confusingly named because, unlike `Future.get()`, it never blocks.¹³ Some of that feedback is reflected in the later changes. Some of it, such as the `get()` name, is part of the API that's now proposed as final.

HOW IT WORKS

The basic shape: open, fork, join, close

Here's the same dashboard method written with the API as it stands in JDK 27, which is exactly what JEP 543 proposes to make final. The first example in the JDK 27 Javadoc has the same shape.¹⁴

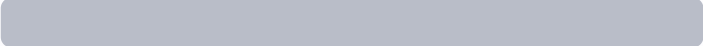
```
// JDK 27 with --enable-preview; proposed final, no flag, in JDK 28.
try (var scope = StructuredTaskScope.open()) {
    Subtask<String> user    = scope.fork(() -> findUser());
    Subtask<String> orders = scope.fork(() -> fetchOrders());
    scope.join(); // throws ExecutionException if either fails
    return new Dashboard(user.get(), orders.get());
}
```

Four things happen, and each has a rule.

1. `open()` creates the scope. The thread that opens it is its **owner** *≈ the thread that opened a scope; only it may join and close it*. The no-argument version uses the default policy: wait for all subtasks, and cancel the rest if any fails.⁷
2. `fork()` starts a subtask, by default in a brand-new virtual thread, and returns a `Subtask` handle straight away.
3. `join()` is the single place where the owner waits. If a subtask fails, the scope is cancelled: every unfinished subtask's thread is **interrupted** *≈ sent Java's standard "please stop" signal, which blocking calls such as `sleep()` and most I/O respond to by throwing `InterruptedException`*, and `join()` throws an `ExecutionException` whose cause is the original error.
4. `close()`, called automatically at the end of the **try-with-resources** *≈ the `try (var x = ...)` form of a try block, which guarantees `x.close()` runs when the block exits, even on an exception* block, makes sure nothing is left running.

Run it against the same failing user service and the difference is immediate. Dan Vega's comparison measured about 209 ms to fail with a scope, against a full second with plain futures.⁸ We saw the same thing on JDK 21 using the older preview API (`ShutdownOnFailure`, compiled with `--enable-preview`): 228 ms, with the orders call interrupted before it finished.

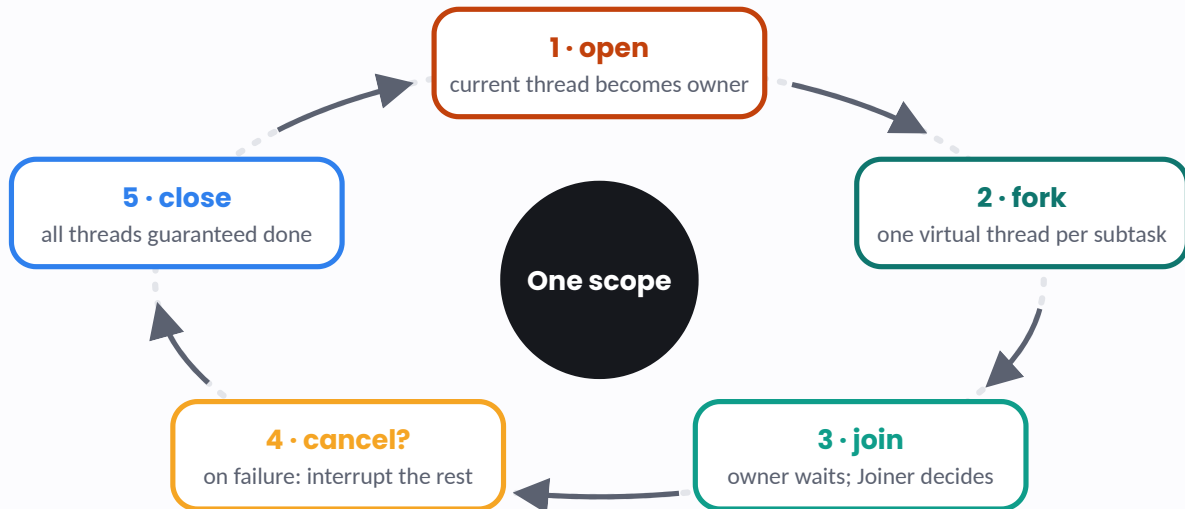
Time until the dashboard call fails (user service fails at 200 ms, orders takes 1 s)

ExecutorService + Future		1,027 ms
StructuredTaskScope		228 ms

Our own single run on JDK 21.0.12 (the scope used the JDK 21 preview API with `--enable-preview`). Dan Vega reports ~209 ms vs ~1 s for the same pattern on JDK 27. Illustrative: the point is the shape, not the exact milliseconds.

One subtlety: cancellation is **cooperative** *≈ it works only if the code being cancelled checks for the signal; Java can't forcibly kill a thread*. Interrupting a thread that's sleeping or waiting on most I/O makes it stop quickly. A task stuck in a tight CPU loop that never checks `Thread.interrupted()` will hold the scope open until it finishes.⁸

One scope's life



Joiners: choosing the policy

The default policy suits "I need all of these". For other shapes you pass a **Joiner** *≈ an object that decides when a scope is done and what `join()` returns or throws* to `open()`. The JDK 27 Javadoc lists the built-in ones:¹⁵

- `Joiner.awaitAllSuccessfulOrThrow()`: the default behaviour. `join()` returns `null` and you read each `Subtask`.
- `Joiner.allSuccessfulOrThrow()`: `join()` returns a `List` of all the results, or throws if any failed.
- `Joiner.anySuccessfulOrThrow()`: returns the first successful result and cancels the rest. That's the "race several replicas, take the fastest" pattern. It throws only if every subtask failed.
- `Joiner.allUntil(predicate)`: waits until everything finishes or your predicate says "enough", and returns the list of `Subtask` handles so you can inspect each one.

```
// Ask three mirrors; first good answer wins, the rest are cancelled.
try (var scope = StructuredTaskScope.open(
    Joiner.<Price>anySuccessfulOrThrow())) {
    mirrors.forEach(m -> scope.fork(() -> m.quote(sku)));
    return scope.join();
}
```

Each `Subtask` ends in one of three states: `SUCCESS`, `FAILED`, or `UNAVAILABLE` (it was cancelled, or didn't finish before a timeout).¹⁵ You can also write your own joiner. It has callbacks `onFork()` and `onComplete()`, either of which can return `true` to cancel the scope, plus `result()` and `timeout()` to produce the outcome. The HappyCoders guide walks through a "best supplier price" joiner that only fails if every supplier fails. Note that a joiner holds state, so you create a fresh one for each scope.⁷

Exceptions and the third type parameter

The JDK 27 round changed two things you'll notice in real code.^{12,16}

First, failures now arrive as a **checked exception** *≈ an exception the compiler forces you to catch or declare in a throws clause*: `java.util.concurrent.ExecutionException`, the same wrapper `Future.get()` has always used. Earlier previews threw a preview-only, unchecked `FailedException`. If you tried an older preview, that `catch` clause is the one thing to update.

Second, `StructuredTaskScope` and `Joiner` gained a third **type parameter** *≈ a placeholder type in a generic declaration, like the `T` in `List`*. The Javadoc declares the API as `sealed interface StructuredTaskScope<T, R, R_X extends Throwable>`, where `T` is the subtasks' result type, `R` is what `join()` returns and `R_X` is the exception `join()` throws.¹⁴ That lets `join()` declare exactly what it throws instead of a vague `throws Exception`. Application code usually never spells the types out, because the compiler infers them. It also lets you pick your own exception, which is handy in a service layer:

```
var joiner = Joiner.<Quote, PricingException>allSuccessfulOrThrow(
    cause -> new PricingException("pricing failed", cause));
try (var scope = StructuredTaskScope.open(joiner)) {
    // fork...; join() now throws PricingException
}
```

That `esf` (exception supplier function) overload exists for all three `...OrThrow` joiners.¹⁵ Because it's a **sealed interface** *≈ an interface that lists exactly which types may implement it (Java 17+)*, you can't write your own `StructuredTaskScope` implementation, only your own `Joiner`.

Failing fast: JDK 21 preview vs the proposed final API

JDK 21 preview (JEP 453)

```
try (var scope = new
    StructuredTaskScope
        .ShutdownOnFailure()) {
    var u = scope.fork(this::user);
    var o = scope.fork(this::orders);
    scope.join().throwIfFailed();
    return new Dash(u.get(), o.get());
}
```

JDK 27 / proposed final (JEP 543)

```
try (var scope =
    StructuredTaskScope.open()) {
    var u = scope.fork(this::user);
    var o = scope.fork(this::orders);
    scope.join(); // ExecutionException
    return new Dash(u.get(), o.get());
}
```

We compiled and ran the left-hand pattern on JDK 21 with `--enable-preview`. The right follows the JDK 27 Javadoc; we couldn't run JDK 27 in our sandbox.

Timeouts and configuration

There's a second `open()` overload that takes a function to adjust the scope's `Configuration`: `withTimeout(Duration)`, `withName(String)` for monitoring, and `withThreadFactory(...)` if you need

something other than default virtual threads.⁷ Since JDK 27 you can configure a scope without supplying a joiner:

```
try (var scope = StructuredTaskScope.open(
    cf -> cf.withTimeout(Duration.ofSeconds(2))
        .withName("checkout"))) {
    // fork, join...
}
```

The timer starts when the scope opens. If it expires, the scope is cancelled and `join()` returns whatever the joiner's `timeout()` method produces. For the built-in `...OrThrow` joiners that's an exception whose cause is a `CancelledByTimeoutException`. For `allUntil()` it's the subtasks collected so far.^{14,7}

The guard rails

Structured concurrency is a set of rules, and the API enforces them.¹⁴

- Only the owner can call `join()` or `close()`. Other threads get a `WrongThreadException`.
- `join()` is called once. Closing a scope that forked but never joined throws `IllegalStateException`.
- Scopes nest. A subtask can open its own scope, and they form a tree. If you close an outer scope before an inner one, the inner ones are closed first and a `StructureViolationException` is thrown.
- Subtasks automatically inherit **scoped values** *≈ a Java 25 feature for sharing an immutable value, such as a request ID, with everything a thread calls, without passing it as a parameter*. If you bind a request ID before opening the scope, every forked subtask (and their children) can read it.⁷

You can see the tree

Because the JVM knows which scope owns which threads, it can show you. Run `jcmd <pid> Thread.dump_to_file -format=json dump.json` (with **jcmd** *≈ the JDK's command-line tool for sending diagnostic commands to a running JVM*) and subtasks appear nested under their scope and owner, instead of as an anonymous list of pool threads.^{7,8} For anyone who has tried to work out which of 400 threads belongs to a stuck request, that's a big quality-of-life change.

What a JSON thread dump shows with nested scopes

Request thread (owner)

opened scope "checkout"

Subtasks: cart, profile, price

virtual threads, children of checkout

Nested scope in "price"

its own subtasks: mirror A, B, C

Nothing outlives its parent

close() waits for every level

Illustrative structure based on the Javadoc's nesting rules and the documented JSON thread-dump output.

WHAT IT MEANS FOR YOU

If you're on JDK 21 or 25 today, nothing breaks. Your `ExecutorService` code keeps working. Structured concurrency isn't replacing executors. It's a better tool for one specific, very common job: "this request needs several things done in parallel, and they all belong to it." Dan Vega's guidance is a useful boundary: it fits subtasks that belong to one operation and should end together, not scheduled jobs, long-lived background work or CPU-heavy number crunching.⁸

The timing question. JDK 28 is a non-LTS release due in March 2027, and the next LTS, JDK 29, follows in September 2027.⁶ If JEP 543 lands in JDK 28 as proposed, then JDK 29 will be the first LTS where structured concurrency is a normal, final API. Writing about the JDK 27 preview, Ron Veen predicted one more preview round in JDK 28 and finalisation only in JDK 29.¹⁶ The proposal moved faster than that.

Where you'll actually meet it. Frameworks are the big multiplier. Spring's Brian Clozel already pointed out in 2025 that structured concurrency lets you compose blocking `RestClient` calls asynchronously without any further changes to the client (see Story 2).¹⁷ Once the API is final, expect framework helpers and libraries to build on it, because they no longer have to ask users to turn on preview features.

In the real world

The open-source media server **streamarr** already runs on JDK 27 with preview features turned on, using `StructuredTaskScope` in its library scanner, refresh loops and artwork downloads. Its tracking issue for JDK 28 is a neat preview of what adoption looks like: remove `--enable-preview` from the build, the tests, the Procfile, the container image and CI. It also notes that each scope has to keep its own policy, for example that one failed item must not cancel its siblings in a scan. That issue is blocked until JDK 28 is GA and a buildpack ships it.²

How to try it today. Install JDK 27, compile with `javac --release 27 --enable-preview` and run with `java --enable-preview`. Preview features only work on the exact JDK version they belong to, so JDK 27 preview code won't run on JDK 26 or 28 builds with the flag.⁷ If JEP 543 lands unchanged, the same source compiles on JDK 28 without the flag.

☀ Watch out for

- **It's a Candidate, not a done deal.** JEP 543 isn't targeted to JDK 28 yet, and the CSR for the implementation hasn't been approved. "No changes" is the proposal, not a guarantee.
- **Old tutorials are wrong.** Anything showing `new StructuredTaskScope.ShutdownOnFailure()`, `FailedException`, `Joiner.awaitAll()` or `onTimeout()` describes an earlier preview. `awaitAll()` was removed in JDK 27; HappyCoders suggests `Joiner.allUntil(_ -> false)` as the equivalent.⁷
- **Subtask.get() doesn't wait.** Unlike `Future.get()` it never blocks; it behaves more like `Future.resultNow()`, so read results only after `join()`.¹³
- **Cancellation is cooperative.** Code that ignores interrupts can keep a scope open.
- **Don't share a joiner** between scopes, and don't reuse one after its scope closes.⁷

WHAT HAPPENS NEXT

The next milestones are the JEP being proposed to target JDK 28 (followed by a short review period), approval of the CSR, and the PR being merged into the JDK mainline. You'll see it in a JDK 28 early-access build after that. The reviewers on PR #32602 are still debating wording, for example whether the docs should say the exception comes from the "earliest" or from "any" failed subtask, which tells you how settled the design is: they're polishing sentences, not semantics.³

📖 Words from this story

Candidate: a JEP accepted, not yet assigned to a release.

CSR: OpenJDK's formal sign-off for public API changes.

Thread: an independent path of execution.

ExecutorService: owns threads and runs tasks you submit.

Future: a handle to a later result; `get()` blocks.

Thread leak: a thread that outlives the code that started it.

Thread dump: a snapshot of every thread in a JVM.

Structured concurrency: tasks started in a block finish before it ends.

Scope: a block that owns concurrent tasks and outlives them.

Subtask: work forked inside a scope, on its own thread.

Platform thread: a Java thread backed by an OS thread.

Virtual thread: a cheap, JVM-managed thread.

Project Loom: the OpenJDK project behind virtual threads.

Incubator: an experimental module for early feedback.

Preview API: complete but not permanent; needs `--enable-preview`.

Owner: the thread that opened a scope.

Interrupt: Java's standard "please stop" signal to a thread.

Try-with-resources: a `try (...)` that always calls `close()`.

Cooperative cancellation: works only if the task checks for it.

Joiner: decides when a scope is done and what `join()` returns.

Checked exception: one you must catch or declare.

Type parameter: a placeholder type, like `T` in `List<T>`.

Sealed interface: restricts which types may implement it.

Scoped value: an immutable value shared down a call chain.

jcmd: the JDK's diagnostic command tool.

STORY 2 RestTemplate is deprecated for removal. Now what?

SPRING FRAMEWORK 7.1 · SPRING BOOT 4.2 · GA IN NOVEMBER 2026 · ~15 MIN READ

If you learned Spring in the Java 8 days, `RestTemplate` was probably one of the first classes you typed. `restTemplate.getForObject(url, User.class)`, a `@Bean` method in a config class, and you were calling other services. It has outlived several generations of "the modern way", and the Spring team has been nudging people away from it, in one form or another, for years. This autumn they stopped hinting. In the Spring Framework 7.1 milestones, `RestTemplate` carries `@Deprecated(forRemoval = true)`, and from November your compiler will tell you so on every build.

WHAT HAPPENED

On 31 March 2026, Brian Clozel of the Spring team opened issue #36574 in the Spring Framework repository, "Deprecate RestTemplate and related types". Its goal was to formally deprecate `RestTemplate` and related classes for removal, to send a stronger signal to the community, with removal planned for Spring Framework 8.0, which isn't scheduled yet. The issue was assigned to the 7.1.0-M1 milestone and is now closed.¹⁸

We checked the source at the `v7.1.0-M2` tag. The class now reads:

```
/**
 * ...
 * @since 3.0
 * @deprecated as of 7.1, in favor of {@link RestClient}.
 * For removal in 8.0.
 */
@Deprecated(since = "7.1", forRemoval = true)
public class RestTemplate extends InterceptingHttpAccessor
    implements RestOperations {
```

The "related types" are a long list. In the same package, `RestOperations`, `RequestCallback`, `ResponseExtractor`, `HttpMessageConverterExtractor`, `ResponseErrorHandler`, `DefaultResponseErrorHandler`, `ExtractingResponseErrorHandler` and `NoOpResponseErrorHandler` are all deprecated for removal as of 7.1, as are `RestGatewaySupport` and `RestTemplateAdapter` in the `support` subpackage.¹⁹

Spring Boot followed. The Spring Boot 4.2.0-M1 release notes, published with the milestone on 20 August, say that "with Spring Framework 7.1 deprecating `RestTemplate` for removal, Spring Boot's supporting infrastructure has been deprecated as well, in favor of `RestClient`". That covers `RestTemplateAutoConfiguration`, `RestTemplateBuilder`, `RestTemplateCustomizer` and the test helpers, and `TestRestTemplate` is deprecated too.^{20,21} In the Boot source at `v4.2.0-M2`, `RestTemplateBuilder` and `TestRestTemplate` are both marked "deprecated since 4.2.0 for removal in 4.4.0".²² Boot's own tracking issue, #51118, simply says that with `RestTemplate` being marked for removal, "our usage of it should be adapted accordingly".²³

Where are we on the calendar? Spring Boot 4.2.0-M2 shipped on 25 September.²⁴ The draft 4.2.0-RC1 release notes were last updated on the project wiki on 8 October, but when we checked the repository on 9 October there was no RC1 tag yet.²⁵ Spring Boot ships new minor versions in May and November, on "the Thursday after the 3rd Monday of the month".²⁶ By that rule, Boot 4.2 and Framework 7.1 should land on **19 November 2026**. That's our calculation; the Spring team hasn't published the exact date.

THE BACKGROUND YOU NEED

Three clients, three eras

An **HTTP client** *≈ code that sends HTTP requests to another service and turns the response into Java objects* is something nearly every backend needs. Spring has shipped three, and it helps to see them side by side.

RestTemplate arrived in Spring Framework 3.0, more than 15 years ago, as Brian Clozel noted in 2025.¹⁷ It's a **template-method API** *≈ an API style where one class offers a ready-made method for each common case, like `getForObject` or `postForEntity`, with the plumbing hidden inside*, the same pattern as **JdbcTemplate**. That worked nicely at first. But every new option meant another overload. Arjen Poutsma, introducing its successor in 2023, said that exposing every HTTP capability through one template class led to "too many overloaded methods".²⁷

WebClient came with Spring Framework 5 and the **reactive** *≈ a non-blocking programming style where you describe a pipeline of operations that run when data arrives, instead of waiting on a thread* stack. It used a **fluent API** *≈ an API where you chain method calls, like `client.get().uri(...).retrieve()`, so each step only offers the options that make sense next* and handled streaming well. At that point, Clozel recalls, the team declared **RestTemplate** "feature complete".¹⁷ The Javadoc spelled it out: "As of 5.0 this class is in maintenance mode, with only minor requests for changes and bugs to be accepted going forward."²⁸ For a while, the Javadoc went further. A note quoted in a Spring Cloud Netflix issue read: "The RestTemplate will be deprecated in a future version and will not have major new features added going forward."²⁹ That wording was later dropped, and for years developers argued about whether **RestTemplate** was "really" deprecated.³⁰

The trouble was that **WebClient** belonged to the reactive stack, and lots of teams with ordinary blocking Spring MVC apps simply kept using **RestTemplate**.

RestClient closed that gap. It shipped in Spring Framework 6.1, which went GA on 16 November 2023 alongside virtual-thread support.³¹ Poutsma described it as a synchronous client with a fluent API like **WebClient**'s, built on **RestTemplate**'s existing "message converters, request factories, interceptors, and other underlying components".²⁷ That last part is why migration is feasible: underneath, both clients use the same machinery.

RestTemplate and RestClient share the same engine room

RestTemplate (deprecated) / RestClient

the API your code calls

Interceptors + initializers

auth headers, tracing, logging

HttpMessageConverters

JSON $\rightleftharpoons$ objects (Jackson), etc.

ClientHttpRequestFactory

JDK HttpClient, Apache, Jetty, Reactor

The network

TCP/TLS to the other service

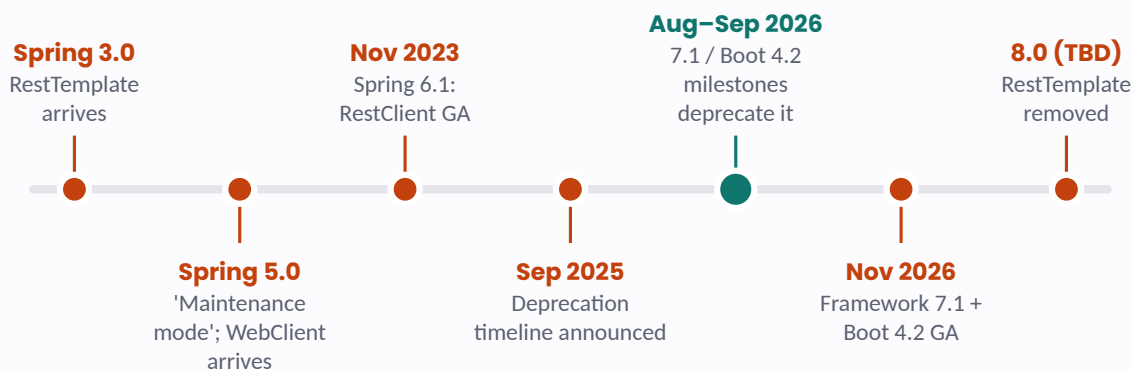
From the RestTemplate Javadoc at v7.1.0-M2: both clients 'share the same infrastructure', but 'RestClient is the focus for new higher-level features'.

The current RestTemplate Javadoc says exactly this: the two share request factories, interceptors, initializers and message converters, "so any improvements made therein are shared as well. However, RestClient is the focus for new higher-level features."¹⁹

The plan, announced a year in advance

The timeline wasn't a surprise. On 30 September 2025, Clozel published "The state of HTTP clients in Spring" and laid out three steps: announce the intent with Spring Framework 7.0 in November 2025; formally mark RestTemplate @Deprecated for removal in 7.1 (provisionally November 2026); and remove it in 8.0, date to be decided. His reasons were that the template-style API can't close the feature gap with RestClient, can't easily take on features such as **API versioning** $\approx$ *sending a version number with each request (in a header, path or media type) so a server can evolve its API safely*, and handles streaming and async composition poorly. He added that, at the current release pace, open-source support for RestTemplate would last until at least 2029.¹⁷ InfoQ's Spring 7 coverage in November 2025 reported the same plan.³²

RestTemplate's long goodbye



Sources: RestTemplate Javadoc; Spring blog (Poutsma 2023, Clozel 2025); Spring 6.1 GA post; Boot 4.2 milestone notes. Boot removes its helpers in 4.4.

HOW IT WORKS

What "deprecated for removal" does to your build

There are two kinds of deprecation in Java. A plain `@Deprecated` produces a warning only if you compile with `-Xlint:deprecation`; most builds never show it. `@Deprecated(forRemoval = true)` is louder: the compiler emits a **removal warning** *≈ a compiler warning, on by default, that you're using an API its authors plan to delete* on every use, with no extra flags. We checked this on JDK 21 with a dummy class carrying exactly Spring's annotation:

```
Use.java:1: warning: [removal] Old in lib has been
    deprecated and marked for removal
```

Add `-Werror`, which many teams use to keep builds clean, and the same code fails with "warnings found and -Werror specified". So the practical effect of upgrading to Boot 4.2 is that a codebase full of `RestTemplate` either gets noisy or stops compiling until you add `@SuppressWarnings("removal")` or migrate. Nothing changes at runtime: `RestTemplate` works exactly as before.

The Boot side of the change

Spring Boot's **auto-configuration** *≈ Boot's mechanism that creates sensible beans for you based on what's on the classpath* gives you ready-made **beans** *≈ objects that Spring creates and manages, and injects wherever you ask for them* for both clients: a `RestTemplateBuilder` and a `RestClient.Builder`. The M1 notes give the direct mappings:²⁰

- Building from the auto-configured `RestTemplateBuilder` → inject `RestClient.Builder` instead.
- A `RestTemplateCustomizer` bean → the equivalent `RestClientCustomizer`.
- `@RestClientTest` with `MockRestServiceServer` → configure a `RestClient.Builder` rather than a `RestTemplateBuilder`.
- `TestRestTemplate` in integration tests → `RestTestClient`, injected with `@AutoConfigureRestTestClient`. It can call a live server or use a mock transport depending on the test, and has built-in assertions.

The reference documentation section on calling REST services no longer covers `RestTemplate` at all.²⁰ Here's what the replacement integration test looks like in Boot's own documentation sources at `v4.2.0-M2`:²²

```
@SpringBootTest(webEnvironment = WebEnvironment.RANDOM_PORT)
@AutoConfigureRestTestClient
class MyRandomPortRestTestClientTests {
    @Test
    void exampleTest(@Autowired RestTestClient restClient) {
        restClient.get().uri("/")
            .exchange()
            .expectStatus().isOk()
            .expectBody(String.class).isEqualTo("Hello World");
    }
}
```

Translating the calls

For application code, the Spring Framework reference has an equivalence table. Each `RestTemplate` method becomes a short fluent chain.³³

The same two calls, before and after

`RestTemplate` (deprecated in 7.1)

```
Pet pet = restTemplate.getForObject(
    url, Pet.class);

ResponseEntity<Pet> res =
    restTemplate.postForEntity(
        url, newPet, Pet.class);
```

`RestClient` (Spring 6.1+)

```
Pet pet = restClient.get().uri(url)
    .retrieve().body(Pet.class);

ResponseEntity<Pet> res =
    restClient.post().uri(url)
        .body(newPet).retrieve()
        .toEntity(Pet.class);
```

Mappings from the Spring Framework reference documentation's `RestTemplate-to-RestClient` table.

A few more rows from that table: `exchange(URI, HttpMethod, HttpEntity, Class)` becomes `method(...).uri(...).headers(...).body(...).retrieve().toEntity(...)`; `delete(URI)` becomes `delete().uri(URI).retrieve().toBodilessEntity()`; and the low-level `execute(...)` with callbacks becomes `method(...).uri(...).exchange(...)`.³³

One important detail: calling `retrieve()` on its own does nothing. The docs say it "is a no-op and returns a `ResponseSpec`", and you have to call a terminal method such as `body()` or `toEntity()` for the request to happen.³³

Error handling moves too

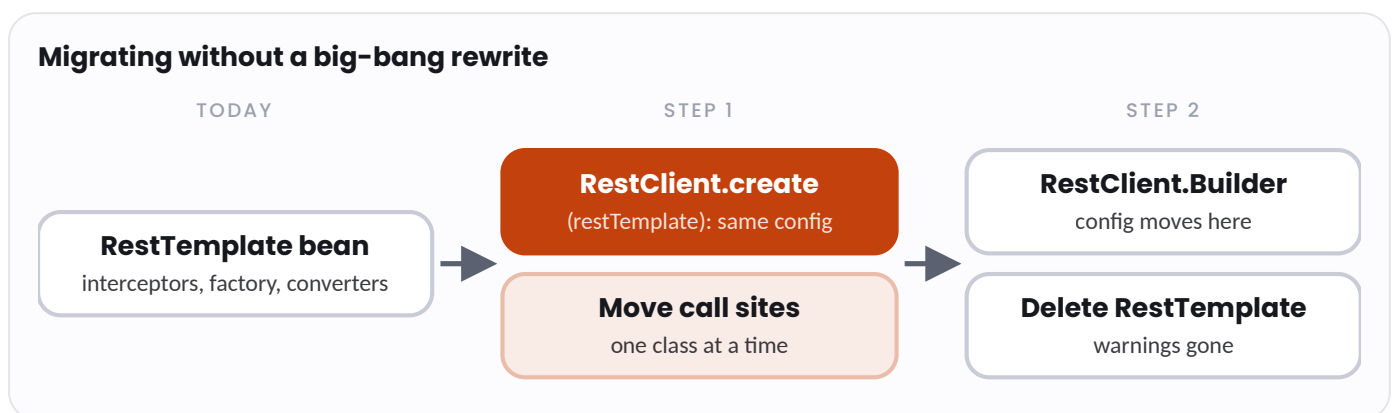
With `RestTemplate`, you customised errors by setting a **`ResponseErrorHandler`** *≈ RestTemplate's hook for deciding which responses count as errors and what to throw*. That interface, and its default implementation, are deprecated in 7.1 as well, in favour of `RestClient.ResponseSpec.ErrorHandler`.¹⁹ Even the `RestClient.Builder.defaultStatusHandler(ResponseErrorHandler)` overload, which exists to ease migration, is now deprecated for removal. The replacement takes a status predicate plus a handler.¹⁹ Per request, you use `onStatus`:

```
String body = restClient.get().uri(url)
    .retrieve()
    .onStatus(HttpStatus::is4xxClientError, (req, res) -> {
        throw new MyCustomRuntimeException(
            res.getStatusCode(), res.getHeaders());
    })
    .body(String.class);
```

By default, 4xx and 5xx responses throw a `RestClientException` subclass, as they did with `RestTemplate`. Status handlers aren't applied when you use `exchange()`, where you handle the raw response yourself.³³ A developer migrating a project that started on Spring Boot 1.5.4 asked whether `HttpStatusException` and `ResourceAccessException` still behave the same. Clozel checked and found no significant differences, because the two clients share the infrastructure that throws them.³⁴

The migration path the Spring team recommends

You don't have to rewrite every call at once. The reference docs describe a two-step path, and the key is a factory method that wraps an existing, fully configured `RestTemplate`.^{33,17}



`RestClient.create(restTemplate)` copies the template's request factory, message converters, interceptors, initializers, URI template handler and error handler into a new `RestClient`.¹⁹ You can switch call sites gradually, and once nothing calls the template directly, you move the configuration onto a `RestClient.Builder`.

☀ Watch out for

- **The default transport is different.** With no other HTTP library on the classpath, `RestClient` uses `JdkClientHttpRequestFactory` (the JDK's `HttpClient`), while `RestTemplate` uses `SimpleClientHttpRequestFactory` (`URLConnection`). The docs warn that this can cause subtle differences in behaviour at the HTTP level.³³ If you wrap with `RestClient.create(restTemplate)`, you keep the old factory.
- **Null bodies and headers.** One migrator found `body()` and `headers()` on the new builder don't accept `null`, and that `additionalInterceptors()` became `requestInterceptor()` / `requestInterceptors()`.³⁴
- **AMQP starter split.** Unrelated to HTTP, but in the same upgrade: Boot 4.2 adds AMQP 1.0 support, so `spring-boot-starter-amqp` users must move to `spring-boot-starter-rabbitmq` (to stay on AMQP 0.9) or `spring-boot-starter-amqp-rabbitmq` (for AMQP 1.0).²⁰
- **Proxy headers.** Framework 7.1's `ForwardedHeaderFilter` now needs to know whether to use the standard `Forwarded` header or `X-Forwarded-*`. Boot defaults to `X-Forwarded-*` when `server.forward-headers-strategy=FRAMEWORK`.²⁰
- **Tomcat redirects.** `server.tomcat.servlet.use-relative-redirects` now defaults to `true`, and three servlet-specific Tomcat properties moved under `server.tomcat.servlet`.³⁵

Going declarative instead

For service-to-service calls, there's an option that skips most of this code: **HTTP interface clients** *≈ Java interfaces annotated with `@HttpExchange` for which Spring generates the implementation at runtime, similar to Feign*. You describe the remote API as an interface and Spring generates a **proxy** *≈ an object generated at runtime that implements an interface by forwarding each call somewhere else* backed by a `RestClient`.³³

```
@HttpExchange(url = "/repos/{owner}/{repo}",
              accept = "application/vnd.github.v3+json")
public interface RepositoryService {
    @GetExchange
    Repository getRepository(@PathVariable String owner,
                           @PathVariable String repo);
}
```

Spring Framework 7.0 added HTTP interface groups via `@ImportHttpServices`, which declares and configures many such clients at once so they can share one `RestClient`.^{17,32}

WHAT IT MEANS FOR YOU

On Boot 3.x, 4.0 or 4.1, nothing is urgent. The `@Deprecated` annotation only arrives with Framework 7.1, the class keeps working, and Clozel's estimate is open-source support until at least 2029.¹⁷ The pressure starts when you move to Boot 4.2 in November, because that's when the removal warnings appear.

Write new code with `RestClient` . It's been available since Spring Framework 6.1, so most teams can start today, before upgrading. The Spring team calls it the best long-term solution, and recommends `WebClient` only when you need reactive APIs or streaming.¹⁷

Look at your tests first. `TestRestTemplate` is often the most widespread use in a codebase, and it's slated for removal from Boot in 4.4, earlier than Framework 8.0 drops `RestTemplate` itself.²² By Boot's six-month cadence, 4.4 would be roughly November 2027, though that's our extrapolation and nothing has been announced.²⁶

Check your libraries. Your own code is only part of it. Shared internal libraries, OAuth helpers and older Spring Cloud integrations often build their own `RestTemplate` . The Spring Cloud Netflix project, for example, had an issue asking to deprecate its `RestTemplate` -based Eureka client support in favour of `WebClient` .²⁹ Those dependencies need to move before Spring Framework 8.0, or they'll break when you upgrade.

In the real world

Automated migration is becoming a product category. Moderne publishes a recipe, `io.moderne.java.spring.boot3.MigrateRestTemplateToRestClient` , that rewrites `new RestTemplate()` to `RestClient.create()` , changes declared types, converts `getForObject` , `postForEntity` , `exchange` and the other common calls, and folds setters like `setErrorHandler` into the `RestClient.Builder` "where safe". It's proprietary (it needs a Moderne subscription and runs from the Moderne CLI), so check the licence before you plan around it.³⁶ Spring Boot itself is the biggest real-world migration: issue #51118 tracks removing its own internal use of `RestTemplate` .²³

WHAT HAPPENS NEXT

The next step is Spring Boot 4.2.0-RC1, whose draft notes already list a new default buildpacks builder (`paketobuildpacks/ubuntu-resolute-builder` , so container images move from Ubuntu 24.04 to 26.04) and a `properties-migrator` option to fail the application on warnings.²⁵ GA follows in November along with Framework 7.1. After that, the open question is when Spring Framework 8.0 arrives. Until a date is set, `RestTemplate` sits in a well-signposted waiting room: still supported, still patched, and no longer something you should be writing new code against.

Words from this story

HTTP client: code that sends HTTP requests and turns responses into Java objects.

Template-method API: one class with a ready-made method per common case (`getForObject` , `postForEntity`).

Reactive: a non-blocking style where you describe a pipeline that runs as data arrives.

Fluent API: chained calls where each step offers only the options that make sense next.

API versioning: sending a version with each request so a server can evolve its API safely.

Removal warning: a compiler warning, on by default, for APIs marked `forRemoval = true` .

Auto-configuration: Boot's mechanism for creating sensible beans based on the classpath.

Bean: an object Spring creates, manages and injects for you.

ResponseErrorHandler: `RestTemplate` 's hook for deciding what counts as an error and what to throw.

HTTP interface client: an `@HttpExchange` interface whose implementation Spring generates.

Proxy: a runtime-generated object that implements an interface by forwarding calls.

On the radar

Also happened, in one line each. Some will get a full deep dive in a future issue.

- **Micronaut 5.2:** versions 5.2.0 and 5.2.1 add an experimental "Micronaut for Python", which processes Python source at compile time, and a new module implementing Jakarta Expression Language.³⁷
- **LangChain4j 1.21.0:** the Java LLM library adds experimental decision-model APIs, including `DecisionModel` , `RoutingChatModel` and `DecisionRouterPlanner` , for routing requests between models.³⁷
- **Google ADK for Java 1.11.0:** Google's agent toolkit adds thread-safe `immutableEvents()` , `addEvent()` and `addEvents()` on `Session` , deprecating the old `events()` method.³⁷
- **OpenXava 8.0:** the low-code framework for business apps reaches GA on Spring Boot, with support for some Jakarta EE 11 specifications and a refreshed look.³⁷
- **A `@note` tag for Javadoc in JDK 28:** a Quality Outreach heads-up describes a proposed tag for highlighted tips and warnings in API docs, inline or block-level, with a configurable heading (JDK-8363700).³⁸
- **CDI 5.0 approved:** the Jakarta EE Specification Committee approved CDI 5.0 as the first specification for Jakarta EE 12.¹

1. [Java News Roundup: New OpenJDK JEPs, CDI 5.0, Spring, Open Liberty, RefactorFirst, ADK for Kotlin](#), InfoQ, 14 Sep 2026.
2. [Move to JDK 28 and adopt the final structured concurrency API without preview features, issue #482](#), streamarr-server, GitHub (checked 9 Oct 2026).
3. [8391460: Implement JEP 543: Structured Concurrency, PR #32602](#), openjdk/jdk, GitHub (open; checked 9 Oct 2026).
4. [Java 28 starts to take shape](#), InfoWorld, 24 Sep 2026.
5. [Inside Java, JDK 28 tag](#), Oracle (targeted-JEP announcements to 6 Oct 2026; checked 9 Oct 2026).
6. [JDK 27 and JDK 28: What We Know So Far](#), InfoQ, 24 Aug 2026.
7. [Structured Concurrency in Java 27 with StructuredTaskScope](#), HappyCoders.eu.
8. [Java Structured Concurrency in JDK 27: A Practical Example](#), Dan Vega, 19 Sep 2026.
9. [Structured Concurrency, Java Platform SE 21 Core Libraries guide](#), Oracle.
10. [JEP 505: Structured Concurrency \(Fifth Preview\)](#), A N M Bazlur Rahman, InfoQ, 12 May 2025.
11. [JEP targeted to JDK 27: 533: Structured Concurrency \(Seventh Preview\)](#), Inside Java, 11 May 2026.
12. [JEP 533: Structured Concurrency in JDK 27](#), InfoQ, 13 May 2026.

13. [Critique of JEP 505: Structured Concurrency \(Fifth Preview\)](#), Adam Warski, SoftwareMill, 25 Sep 2025.
14. [StructuredTaskScope Javadoc, Java SE 27 \(ad-hoc build\)](#), Alan Bateman, cr.openjdk.org.
15. [StructuredTaskScope.Joiner Javadoc, Java SE 27 \(ad-hoc build\)](#), cr.openjdk.org.
16. [JEP 533: Structured Concurrency Gets Sharper in JDK 27](#), Ron Veen.
17. [The state of HTTP clients in Spring](#), Brian Clozel, Spring blog, 30 Sep 2025.
18. [Deprecate RestTemplate and related types, issue #36574](#), spring-projects/spring-framework, opened 31 Mar 2026, milestone 7.1.0-M1.
19. [spring-web org.springframework.web.client sources at tag v7.1.0-M2](#) (RestTemplate.java, RestClient.java, ResponseErrorHandler.java and others), read 9 Oct 2026.
20. [Spring Boot 4.2.0-M1 Release Notes](#), Spring Boot wiki.
21. [Spring Boot 4.2.0-M1 available now](#), Phil Webb, Spring blog, 20 Aug 2026.
22. [Spring Boot sources at tag v4.2.0-M2](#) (RestTemplateBuilder.java, TestRestTemplate.java, MyRandomPortRestTestClientTests.java), read 9 Oct 2026.
23. [Deprecate RestTemplate support, issue #51118](#), spring-projects/spring-boot.
24. [Spring blog index: "Spring Boot 4.2.0-M2 available now"](#), 25 Sep 2026 (checked 9 Oct 2026).
25. [Spring Boot 4.2.0-RC1 Release Notes \(draft\)](#), Spring Boot wiki, last updated 8 Oct 2026; no v4.2.0-RC1 tag in the repository as of 9 Oct 2026.
26. [Supported Versions](#), Spring Boot wiki.
27. [New in Spring 6.1: RestClient](#), Arjen Poutsma, Spring blog, 13 Jul 2023.
28. [RestTemplate Javadoc, Spring Framework 5.3.6](#), docs.spring.io.
29. [Deprecate RestTemplate eureka client support replace with WebClient, issue #3736](#), spring-cloud/spring-cloud-netflix.
30. [Spring RestTemplate won't Deprecate](#), velog (Korean), updated 25 Aug 2024.
31. [Spring Framework 6.1 goes GA](#), Juergen Hoeller, Spring blog, 16 Nov 2023.
32. [Spring Framework 7 and Spring Boot 4](#), Karsten Silz, InfoQ, 21 Nov 2025.
33. [REST Clients, Spring Framework reference documentation \(7.0.x\)](#), docs.spring.io (checked 9 Oct 2026).
34. [Improve RestTemplate to RestClient migration guide, issue #35620](#), spring-projects/spring-framework.
35. [Spring Boot 4.2.0-M2 Release Notes](#), Spring Boot wiki.
36. [Migrate RestTemplate to RestClient \(recipe catalog\)](#), Moderne docs.
37. [Java News Roundup: JobRunr 9, OpenXava 8, Quarkus, LangChain4j, JNoSQL, Introducing Lathe](#), InfoQ, 5 Oct 2026.
38. [Quality Outreach Heads-up - JDK 28: Rich JavaDoc Notes](#), Billy Korando, Inside Java, 23 Sep 2026.